Project Documentation: Survey Feedback Analyzer

This document provides a comprehensive overview of the implementation logic, structure, and functional workflows designed for the **Survey Feedback Analyzer** assignment submission.

1. Code Workflow & Architectural Overview

The program is entirely written in modular Python syntax without external library dependencies, satisfying all execution constraints. It follows five structured steps:

• **Step 1: Preloaded Feedbacks** – Houses the base feedback_data dictionary containing historical test rows.
• **Step 2: User Input System** – Asks the operator dynamically for additional feedback loops, auto-incrementing serial indices beginning from index 11.
• **Step 3: Text Cleaning Engine** – A functional engine executing multi-layered transformations: forces lowercasing, removes targets (., !, ?, ,), and converts consecutive whitespace into single spaces via ' '.join(text.split()).
• **Step 4: Insight Generator** – Uses case-insensitive sub-string filtering via a localized search function.
• **Step 5: Advanced Analytics Suite** – Computes structural statistics including aggregate mean ratings, tokenizes corporate vocabulary matrices to find unique tags, tracks the longest block length, and applies a native zip() sorting scheme to list results from highest to lowest score.

2. Sample Execution Outputs

Below is a conceptual visualization of the final summarized tabular metrics generated under Step 5:

Metric	Value / Output
Average Rating	3.60 (Variable depending on user entries)
Longest Feedback Comment	"poor response and poor handling of issue" (8 words)
Sample Core Word Matches	good: 3 matches | poor: 2 matches | excellent: 1 match

3. Core Cleaning Logic Code Snippet

```
def clean_text(text):
    text = text.lower()
    for punc in ['.', ',', '!', '?']:
        text = text.replace(punc, '')
    return ' '.join(text.split())
```

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.